

Kosha

A fee ledger for an Indian college: payments, a mock payment gateway, receipts, concessions, reversals and gateway settlement reconciliation, on an append-only ledger. With five AI assistants built on LangChain and LangGraph, none of which can move money.

Contents

1. Summary
2. Problem understanding
3. Assumptions
4. Architecture
5. Data model
6. Payment lifecycle
7. Reconciliation
8. Engineering decisions and trade-offs
9. Edge cases and how each is handled
10. Validation and testing
11. Product walkthrough
12. AI features
13. Limitations and future work
14. AI usage report

Figures and tables are numbered. Amounts are in Indian rupees (₹), dates and times in IST. Every figure quoted from the system comes from the demo seed described in section 3.

Candidate Darshan

Date 29 September 2026 (AI features added; first edition 24 September)

1. Summary

Colleges collect fees through several channels at once: a cash counter, bank transfers, and online payments through a gateway that sometimes answers late or not at all. The hard part is not taking the money; it is being able to say, for any student on any day, exactly what they owe, what they paid, and where each rupee went, and proving it against the bank.

Kosha is a fee ledger that does that. An accountant opens a student's statement, records a payment and sees before submitting which installments it will clear. Online payments go through a mock gateway that can succeed, fail or time out. Every successful payment gets a sequential receipt. A daily settlement file from the gateway is uploaded and matched against what was recorded, and every mismatch has a resolution with a name and a note against it. Admins can apply concessions and reverse payments; nothing is ever edited or deleted.

The three decisions that matter most

An append-only ledger. Balances are the sum of ledger entries: demand, concession, payment and reversal. Database triggers reject UPDATE and DELETE on the ledger and on payment allocations. A reversal is a new positive entry, so the history of every account can always be read back, and a reversed payment reopens its installments without deleting anything.

Money rules live in the database, in one transaction. Each money-changing operation is a single PostgreSQL function that locks the student row, checks the rules, writes the payment, events, ledger entry, allocations, receipt number and audit line together, or nothing at all. Constraints and triggers enforce the same rules even if the API is bypassed. Idempotency keys stop double submits.

Reconciliation is a first-class flow, not a report. The gateway file is parsed and validated, matched by a pure, unit-tested function into four buckets, stored as a run, and each exception is resolved through a database function that records who resolved it and why. "Mark as paid" confirms the pending payment in the same transaction.

What was built

- Students list, and a passbook-style statement with running balance, fee-head progress, installments and payments.
- Record payment (cash, UPI, card, bank transfer) with a live allocation preview and a visibly marked gateway simulator; check status; mark as failed; reverse; concessions.
- Payment detail with a state track, allocations, ledger entries and audit trail; printable A5 receipts with the amount in words.
- Reconciliation upload, the four buckets and their resolutions; a dashboard; a readable audit log; simulated roles (admin, accountant, student) enforced by the server.
- Five AI features, each checked in code and decided by a person (section 12): Reconciliation Copilot, Ask Kosha, smart import, messages to parents (English, Kannada, Hindi), daily brief.
- 192 unit, 13 integration, 3 end-to-end tests, 21 AI evaluations, a nine-check ledger script.

2. Problem understanding

Who uses it

User	What they need
Accountant	Record counter and bank payments quickly and correctly; know which installment a payment cleared; chase pending online payments; reconcile against the gateway every day.
Admin (bursar, principal's office)	Approve concessions, handle reversals (bounced transfers, chargebacks), see what is collected and what is overdue, answer audit questions.
Student or parent	See an honest statement, pay online, get a receipt, and know that a payment stuck at the gateway has not been lost.

What actually goes wrong with fee collection

Problem	What it looks like	How Kosha responds
Lost payments	A parent paid by UPI, the gateway timed out, and the college shows nothing. Or a transfer arrives with no record at the counter.	Timed-out payments stay <i>pending</i> with a gateway reference until check status or reconciliation settles them.
Double charges	A slow form is submitted twice; two receipts for one payment.	An idempotency key per form; the database returns the existing payment on a repeat.
Ambiguous pending states	"Did it go through?" Nobody knows, so it is recorded twice or not at all.	An explicit state machine; only SUCCESS touches the ledger.
Disputes	"I paid tuition, why does it say hostel is due?"	Allocation is a fixed, visible rule (oldest due date first), shown before submitting and on every receipt.
Silent corrections	Someone edits last month's entry to make totals match.	Append-only ledger; corrections are new entries with reasons.
Audit questions	"Who reversed this, and why?"	Every money function writes an audit line with actor, action and details.
Bank vs books	Gateway settles a different amount, or never settles a payment we marked paid.	Reconciliation buckets exceptions; mismatches are flagged, never auto-fixed.

3. Assumptions

1. **Roles are simulated.** A role switcher sets a cookie; the server enforces one permission matrix. Real login and per-user row-level security are out of scope for a take-home, but the permission checks sit exactly where real auth would plug in.
2. **One institution, one academic year (2026-27), INR only.** This keeps the data model focused on the ledger; multi-tenancy and currencies are future work.
3. **The payment gateway is a mock.** A "simulate outcome" control (succeed, fail, time out) stands in for the provider; a small table holds what the gateway "knows", so check status and settlement have something to ask.
4. **A timed-out payment usually did go through.** The mock gateway reports success when asked later, which is the common real-world case; staff can still mark a pending payment as failed.
5. **Settlement is T+1.** A file whose settlement dates run from D1 to D2 covers payments made from D1-1 to D2-1; only payments in that window can be "missing in settlement".
6. **Settlement amounts are gross.** The file's amount is compared directly with the payment; gateway fees (MDR) are not modelled.
7. **Fee demand for the whole year is raised on 1 July,** as installments per fee head per term. There is no fee-structure editor.
8. **Allocation is oldest due date first,** fee-head order breaking ties (tuition, hostel, exam, library). Overpayment is kept as an advance and applied automatically when something reopens.
9. **Receipt numbers are one global sequence** with the academic year as a prefix (KSH/2026-27/000123). Gaps are possible if a transaction rolls back after taking a number; this is normal for database sequences and is noted.
10. **The demo is anchored to 24 September 2026.** Seed dates are fixed so gateway references and the sample settlement file always line up after a reset.
11. **Amounts above ₹10,00,000 in one payment are refused** as a guard against typing errors; it is a sanity limit, not a business rule.

The demo data

60 students across B.Tech CSE, BCA and B.Com, 408 installments (4 fee heads × 2 terms; hostel for about 40%), 66 payments over about 45 days, and 469 ledger entries. Seven named students each show one scenario: fully paid (Ananya Rao), partly paid (Rohan Kulkarni), overdue with nothing paid (Vikram Singh), merit concession (Priya Nair), a returned bank transfer that was reversed (Arjun Mehta), a UPI payment stuck in pending (Sneha Iyer), and an advance from overpaying (Karthik Reddy). All history after the fee demand is written through the same database functions the app uses.

4. Architecture

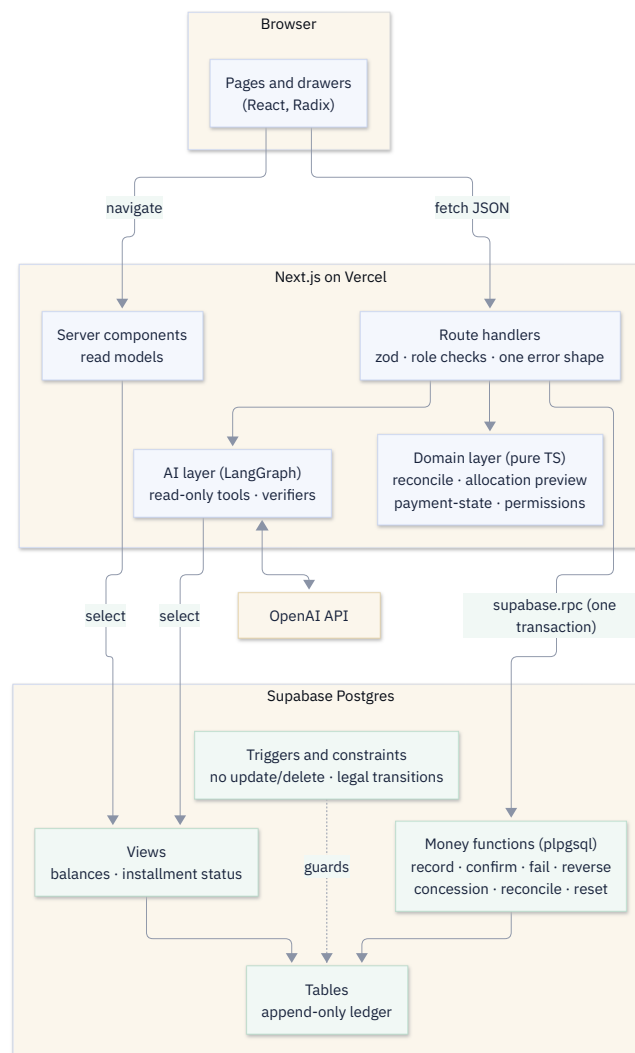


Figure 1. System architecture. The browser never talks to the database; the service-role and OpenAI keys live only in server code.

Why this stack

- **One codebase.** Next.js serves the pages, the API routes and the server-rendered read models, so types are shared from the database row to the button label.
- **Managed Postgres.** Money needs real transactions, row locks, constraints, sequences and triggers. Supabase provides a Postgres with those features and nothing to operate.
- **One-click deploy.** Vercel builds the repository on push; the only configuration is three environment variables, plus `OPENAI_API_KEY` for the AI features.

Where each rule lives

Layer	Responsibility
Database functions	Every money rule: locking, idempotency, allocation, receipts, ledger, audit. The source of truth.

Layer	Responsibility
Constraints and triggers	Defence in depth: append-only tables, legal transitions, sign of ledger amounts, one ledger entry per source row, allocations can only point at the same student's installments.
Grants	The API's service role may read tables and execute the public functions, nothing else. Even a leaked key cannot UPDATE the ledger.
Route handlers	zod validation, the permission matrix, ownership checks for the student role, mapping database errors to readable messages.
AI layer (section 12)	LangGraph agents with read-only tools, and verifiers that check every figure and reference before anything is shown. Proposals only; acting on one goes through the database functions above.
Domain layer (pure TS)	Reconciliation matching, allocation preview, statement building, audit sentences, the state machine for the UI. All unit tested.

5. Data model

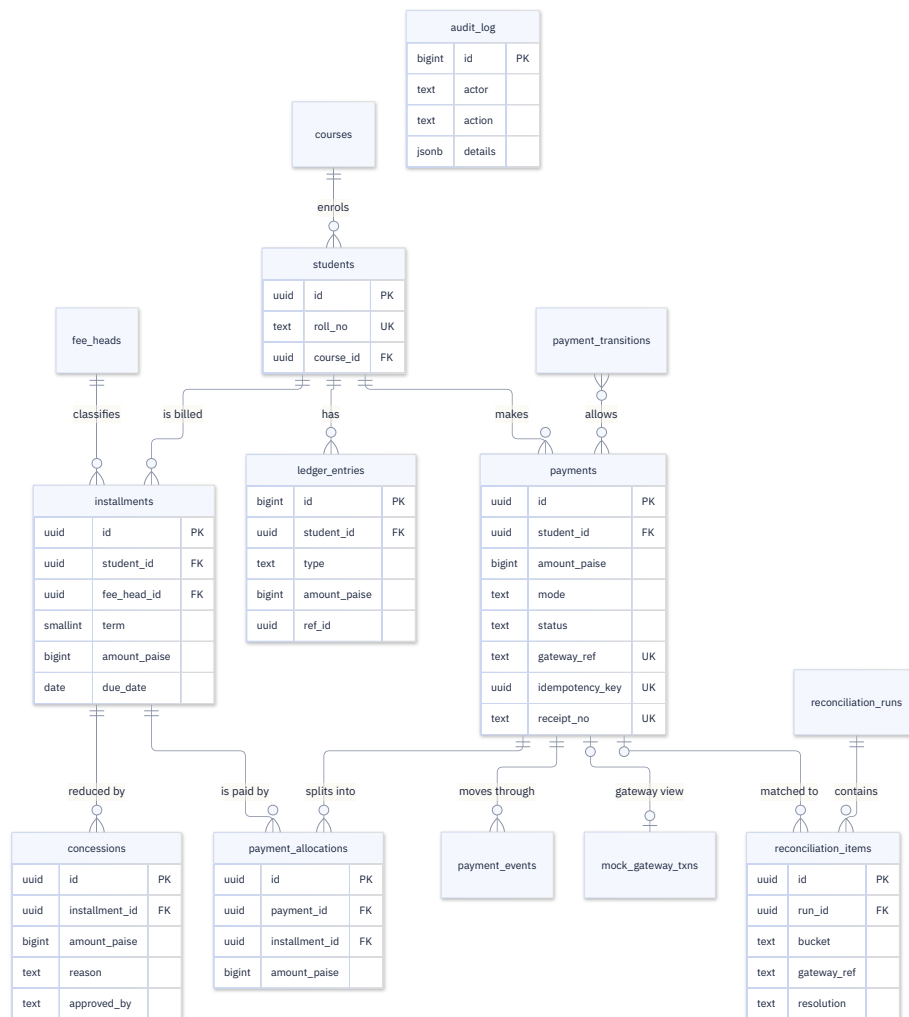


Figure 2. Entity relationships (key columns only). `audit_log` refers to records by entity and id rather than a foreign key, so it can describe any table.

Why money is stored in paise

Floating-point numbers cannot represent most decimal fractions exactly: $0.1 + 0.2$ is not 0.3 . A fee system adds, splits and compares amounts constantly, and a one-paise drift is a reconciliation exception. Every amount is therefore an integer number of paise: `BIGINT` in Postgres and a safe integer in TypeScript. Rupees appear only when formatting for display (`formatINR`) and parsing user input (`toPaise`, which rejects more than two decimals and never goes through floating point).

Why the ledger is append-only

A balance that can be edited cannot be trusted or explained. In Kosha the balance is the sum of ledger entries, each of which points at the row that caused it: a `DEMAND` at an installment, a `CONCESSION` at a concession, a `PAYMENT` and possibly a `REVERSAL` at a payment. Triggers reject `UPDATE` and `DELETE`, a check constraint forces the sign to match the type, and a unique index allows at most one entry of each type per source row, which is what makes "confirm twice"

harmless. The verification script cross-checks the ledger against balances computed independently from the domain tables.

6. Payment lifecycle

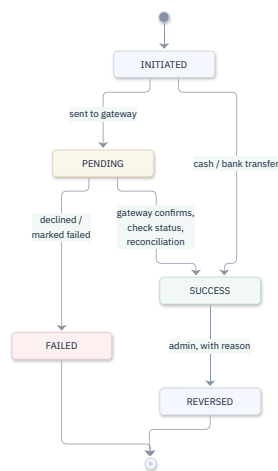


Figure 3. Payment state machine. Any other move is rejected by a database trigger; the same table in TypeScript is tested against the SQL.

Rules

- Only **SUCCESS** writes the ledger, allocates and issues a receipt.
- **Cash and bank transfer** go straight to **SUCCESS**; **UPI and card** pass through **PENDING**.
- **Time out** leaves it **PENDING** with its gateway reference, not counted in the balance.
- A **failed** attempt is kept; a retry is a new payment with a new key.
- **Reversal** (admin, with reason) adds a **REVERSAL** entry and reopens the installments it paid.

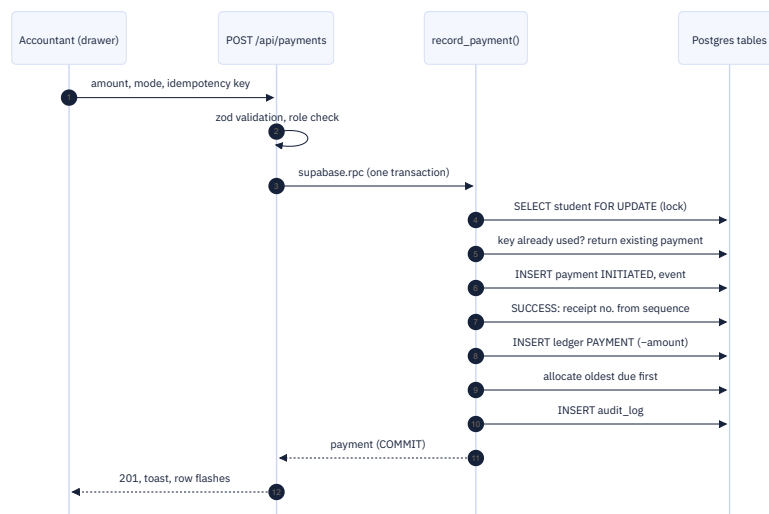


Figure 4. Recording a cash payment. Everything between the lock and the commit happens in one transaction or not at all.

Idempotency, locking and allocation

Idempotency. Each opening of the payment drawer creates a UUID; the column is UNIQUE and is checked after the student lock, so double clicks, retries and parallel submits yield one payment (repeats get it back with HTTP 200). A reused key with a different amount is refused.

Locking. Every money function locks the student row first, then the payment, then the reconciliation item (a fixed order avoids deadlocks), so two payments for one student cannot allocate the same rupee twice.

Allocation. After any success or reversal, unallocated money is applied to open installments, oldest due date first. "Paid" counts only allocations of payments that are *currently* SUCCESS, so a reversal reopens installments without deleting rows; the rest is an advance.

7. Reconciliation

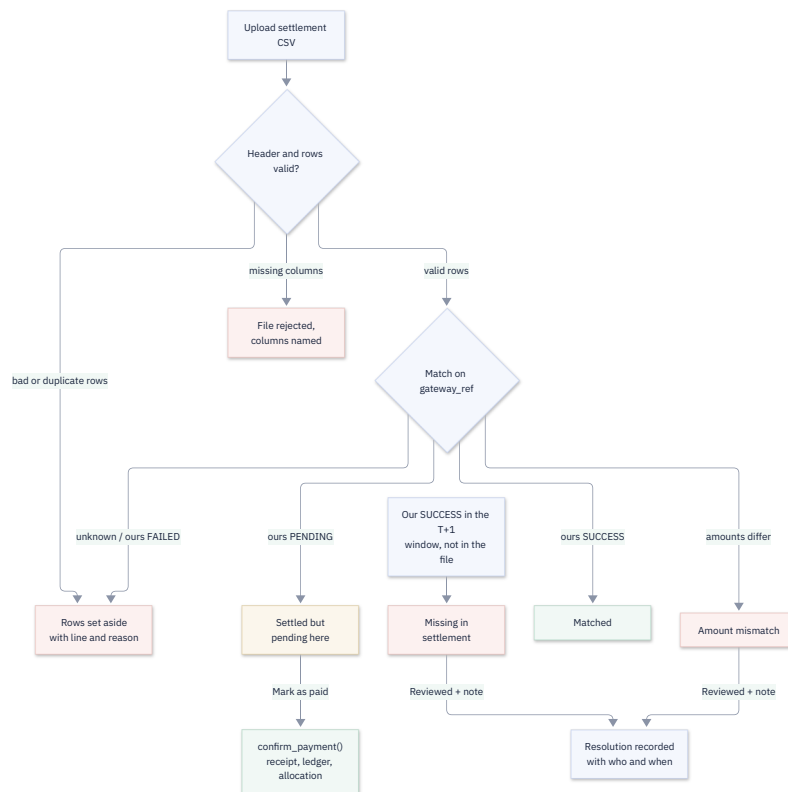


Figure 5. Reconciliation flow. Matching is a pure TypeScript function; storing and resolving go through database functions.

The four buckets, with the sample file

The sample (public/samples/settlement_sample.csv , 36 rows) was generated from the seed with deliberate discrepancies. Against a fresh reset:

Bucket	Rule	Example from the sample	Resolution
Matched (33)	Reference and amount agree; we have it as SUCCESS (or later reversed).	MGW7300000002, Rohan Kulkarni, ₹82,500.	None needed.
Amount mismatch (1)	Reference found, amounts differ. Takes precedence over status.	MGW7300000023, Lakshmi Narayanan: file ₹62,000, recorded ₹62,500 (short capture).	Mark reviewed with a note. Never auto-corrected.
Settled but pending here (2)	The gateway settled it; we still have it PENDING.	MGW7300000005, Sneha Iyer, ₹86,500 (timed out 21 Sep); MGW7300000008, ₹76,000.	Mark as paid: confirms the payment in one transaction.
Recorded here, missing in settlement (1)	Our online SUCCESS inside the file's settlement window, absent from the file.	MGW7300000039, Sowmya Raghavan, ₹40,000 (paid 10 Aug).	Mark reviewed after asking the gateway.

Rows that fit no bucket are listed as "set aside" with the file line and a reason (malformed, duplicate, non-settlement status, unknown reference, or settled but failed here). Each run stores its totals, rejected rows and a file hash, so a repeated upload is flagged.

8. Engineering decisions and trade-offs

Decision	Alternative considered	Why this choice	Cost
Money rules in plpgsql functions	Business logic in TypeScript with a transaction client	One round trip, one transaction, row locks and constraints right next to the data; works through supabase-js without a connection pool of our own.	Two languages; SQL is harder to unit test (covered by integration tests).
Append-only ledger enforced by triggers	Mutable balance column on students	History is always explainable; corrections are visible; balance can be cross-checked.	Reports must sum; reversals need linking in the UI.
Integer paise	numeric rupees	Exact arithmetic in both SQL and TypeScript; no rounding rules to agree on.	Formatting and parsing at every edge.
"Paid" derived from allocations of SUCCESS payments	Delete or negate allocations on reversal	Reversal reopens installments without touching allocation rows; one query answers "what did this payment pay".	Slightly more complex views.
Automatic re-allocation of advance	Manual "apply advance" action	A student never has both spare credit and an open installment (checked by the verification script).	An old receipt can gain a line when an advance is applied later.
Reconciliation matching in pure TypeScript	Matching in SQL	Easy to unit test every edge case (39 tests); readable rules.	The system payments are loaded into memory (fine at this scale).
Service role limited by grants	Full service-role access	A leaked key or a bug in a route cannot write the ledger directly.	Every write needs a function.
Simulated roles via a cookie	Supabase Auth	Out of scope for the assignment; the permission matrix is where real auth would plug in.	Not secure for real use; stated as an assumption.
Fixed demo calendar	Seed relative to today	Gateway refs, receipts and the static sample CSV always line up after a reset.	The 30-day trend thins out if viewed long after 24 Sep 2026.
Server components read, routes write	Client-side data fetching everywhere	Statements render fully on the server; after a change the page re-renders from the database, the single source of truth.	A full refresh of the page data after each action.

9. Edge cases and how each is handled

Case	Handling	Proved by
Double submit	Idempotency key per form open (UNIQUE), checked after the student lock; repeats return the same payment. Submit button disabled while in flight.	Integration (serial and 3 in parallel)
Partial payment	Allocated oldest due first; the last installment it touches is partly paid. The preview says "part of ... ; ₹X will still be due".	Unit, E2E, seed (Rohan)
Overpayment	Never rejected; the remainder is an advance (negative balance), shown as "Advance", used when anything reopens.	Unit, seed (Karthik)
Concession after partial payment	Capped at demand – concessions – paid for that installment; checked in the form, zod and the database.	Integration
Reversal of an allocated payment	REVERSAL entry; allocations stop counting; installments reopen; spare advance re-applied. Second reversal refused.	Integration, E2E
Stuck pending	Shown on the statement and dashboard; resolved by check status, reconciliation "mark as paid", or "mark as failed" with a reason.	Integration, E2E
Confirm racing itself	Row lock plus one-entry-per-source unique index: exactly one confirm succeeds, the other gets "already marked as paid".	Integration
Amount mismatch	Own bucket; both amounts and the difference shown; only "reviewed with a note" is allowed.	Unit, manual
Duplicate refs in the CSV	Identical copies counted once (others listed as duplicates); copies with different amounts all set aside, because nobody can know which is right.	Unit
Malformed CSV rows	Missing columns reject the file; bad rows (wrong field count, bad amount, date or status) are set aside with the line number; blank lines ignored; BOM and CRLF handled.	Unit
Same file uploaded twice	New run, flagged with a link to the earlier run; the dashboard lists each open item once; resolving an already-confirmed payment is harmless.	Manual
Parallel payments	Student row lock serialises them; allocations never exceed demand.	Integration (6 in parallel)
Illegal state transitions	Trigger checks every status change against <code>payment_transitions</code> ; functions give specific messages first.	Unit (all 25 pairs), verify script
Zero or negative amounts	Rejected by <code>toPaise</code> , zod and CHECK constraints (<code>amount_paise > 0</code>).	Unit, integration
Wrong role	UI hides the action; the API answers 403 with the role named; SQL functions check the actor again for reverse, concession and student payments.	Unit (matrix), E2E

10. Validation and testing

Validation at every boundary

Input is validated three times with the same rules: in the form (inline messages), by zod at the route handler (422 with the field name), and in the database by the function and by CHECK constraints. Database errors carry a machine code in the Postgres HINT, which the API maps to an HTTP status and a readable message ("This payment was already reversed.").

What the tests cover

Suite	Tests	Covers
Unit (Vitest)	192	AI verifiers (made-up figures, unknown evidence, forbidden actions, undisclosed duplicates, message language, script and gender), date periods, the settlement-file converter and its sampling, daily-brief signals and review; Payment state machine, every legal and illegal pair, and that the TypeScript table equals the SQL one; reconciliation bucketing, duplicate refs, bad rows, amount formats, T+1 window, CSV parsing (BOM, CRLF, quoted amounts, field counts); <code>toPaise</code> , <code>formatINR</code> , amount in words; allocation preview; statement builder; permission matrix; dates in IST; error mapping.
Integration (Vitest, real database)	13	Same idempotency key twice and three times in parallel; key reuse with another amount; timeout then confirm writes exactly one ledger entry; confirm racing itself; failed payment writes nothing; reversal reopens the installment; double reversal and non-admin reversal refused; concession above remaining refused, exactly remaining accepted; six parallel payments allocate correctly; the API role cannot write the ledger; the verification script passes afterwards.
End to end (Playwright)	3	Record a cash payment (prefill, preview, validation, receipt, balance and row update); an online payment times out and a settlement file resolves it through "mark as paid"; an admin reverses a payment and the installment reopens, an accountant has no such action, a second reversal returns 409.
AI evaluations (real model)	21	Copilot 4, Ask Kosha 8, smart import 2, parent messages 4, daily brief 3; right answers computed independently from the database (section 12.6).
Ledger verification (SQL)	9	Balance = SUM(ledger) for every student; allocations + concessions $\leq$ demand; allocations $\leq$ payment; ledger entries match payment status; one DEMAND per installment; no spare credit next to an open installment; last event matches status; ledger rejects UPDATE and DELETE; FAILED $\rightarrow$ SUCCESS rejected.

Latest results

Command	Result
<code>npm run typecheck · npm run lint</code>	No errors, no warnings
<code>npm run test:unit</code>	Test files 11 passed · Tests 192 passed

Command	Result
<code>npm run eval:ai</code>	21 passed: Copilot 4/4 · Ask Kosha 8/8 · import and messages 6/6 · brief 3/3
<code>npm run e2e</code>	3 passed (1.3 min)
<code>npm run db:reset</code>	All nine ledger checks passed (2.7 s)
<code>npm run build</code>	Compiled successfully

11. Product walkthrough

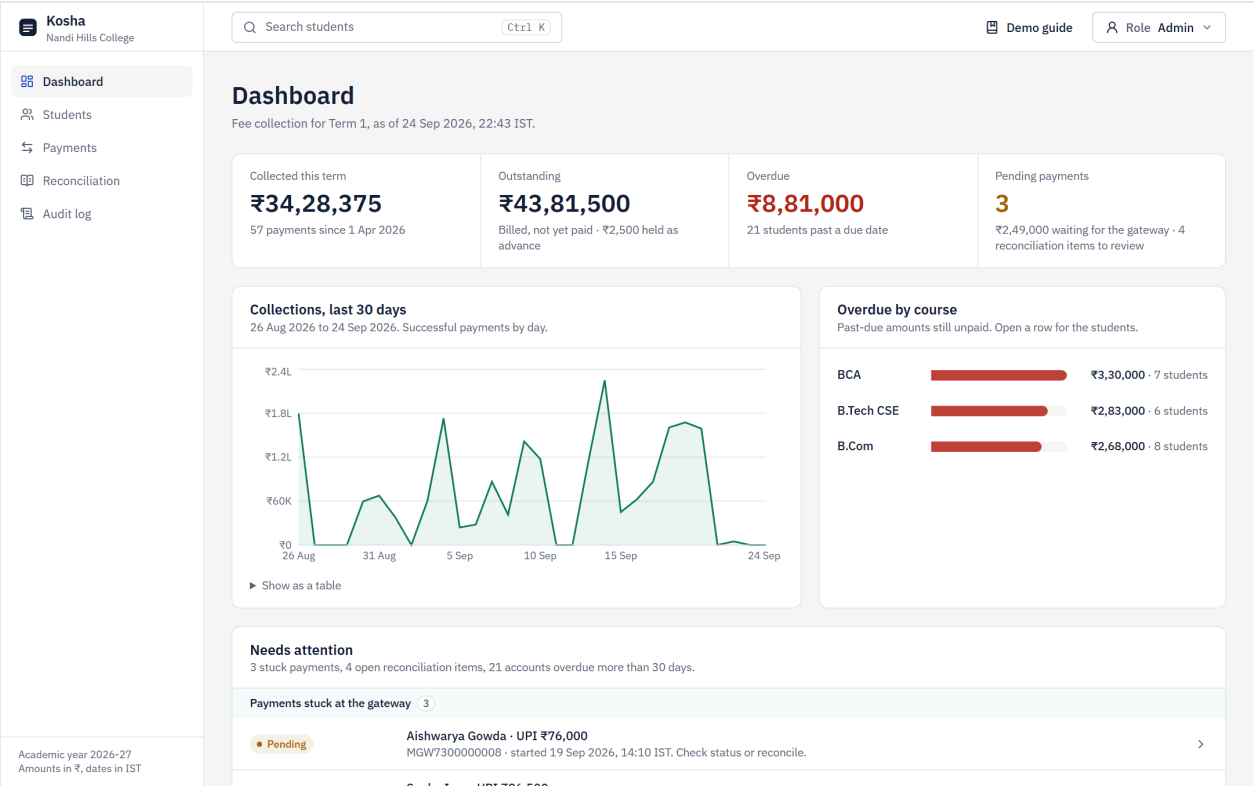


Figure 6. Dashboard. One summary strip (collected, outstanding, overdue, pending), a 30-day collection trend, overdue by course, and a needs-attention list linking to each stuck payment, open reconciliation item and long-overdue account.

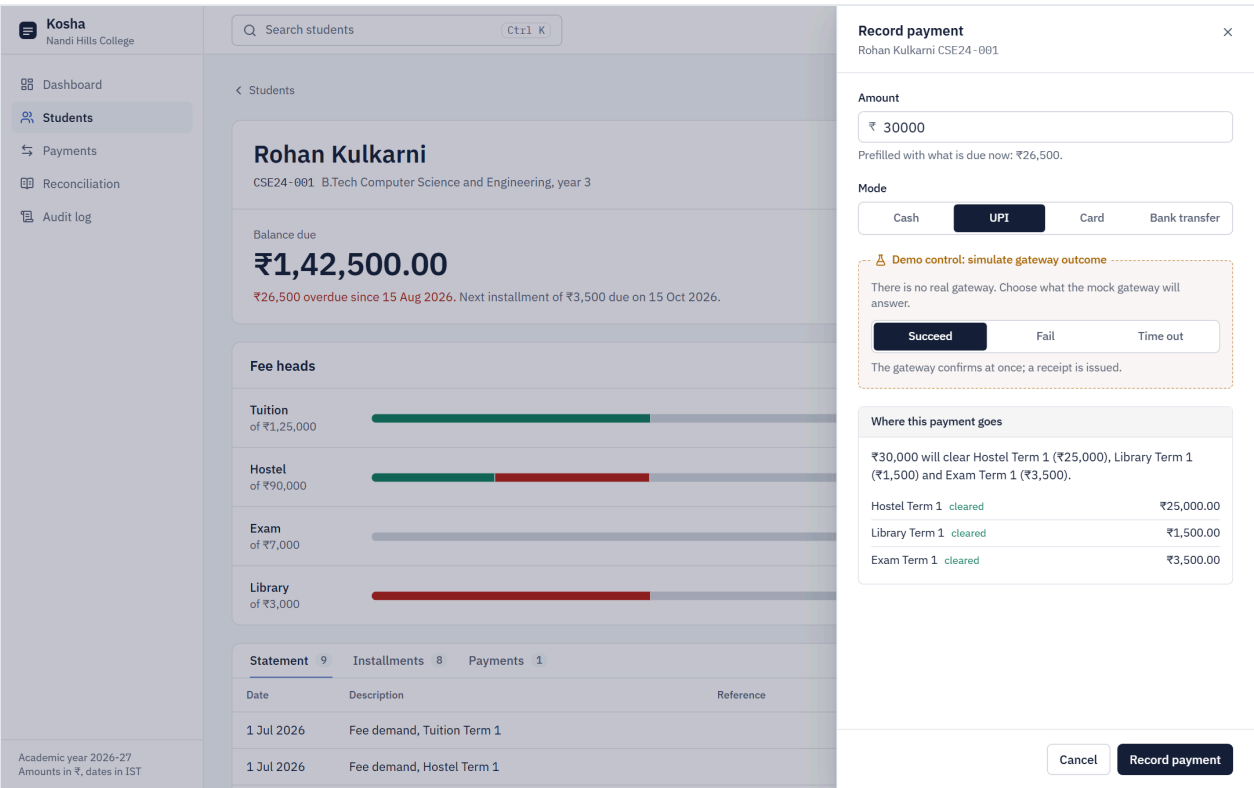


Figure 7. Record payment opens in a drawer so the statement stays visible. The gateway simulator is marked as a demo control; the preview states where the money goes before anything is submitted.

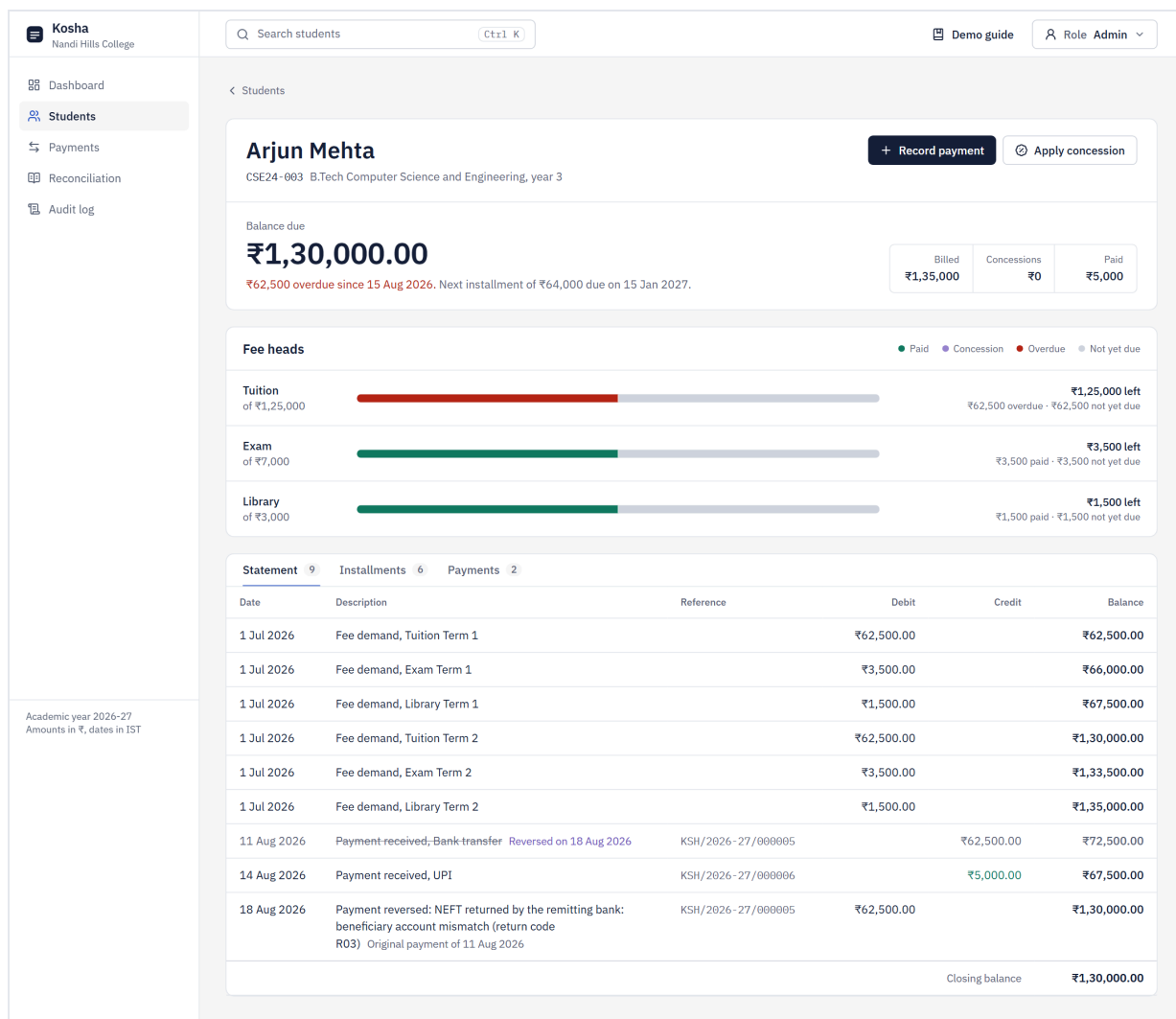


Figure 8. The statement, the signature screen. The balance is the largest figure with a plain sentence under it; fee-head bars show paid, overdue and not yet due; the passbook keeps a running balance. Arjun's returned bank transfer stays visible, struck through and linked to its reversal.

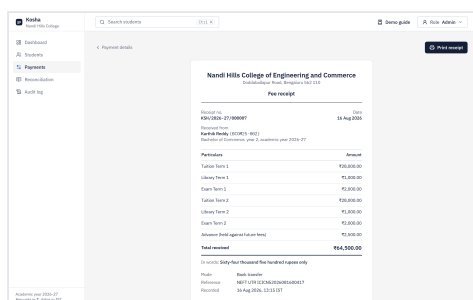


Figure 9. A5 receipt with the amount in words.

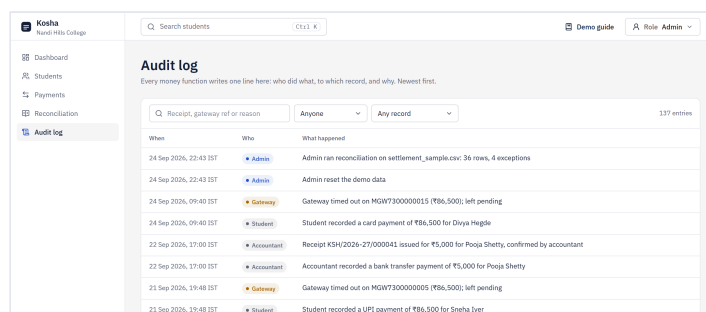


Figure 10. Audit log: one readable line per money function.

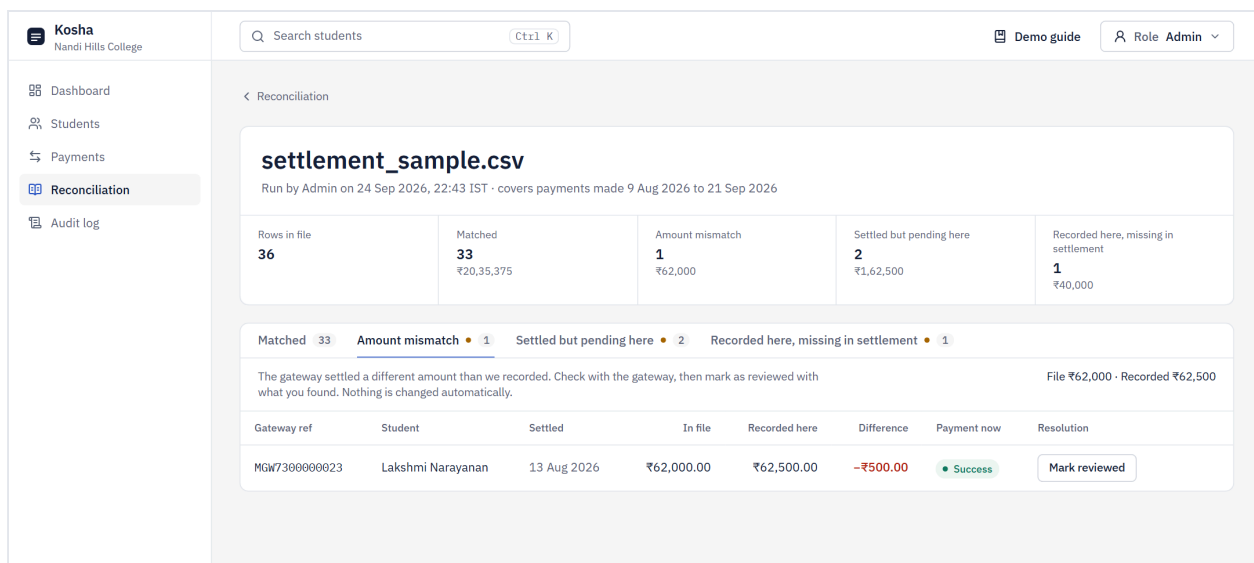


Figure 11. A reconciliation run of the sample file: four buckets with counts and amounts; each exception carries its resolve action.

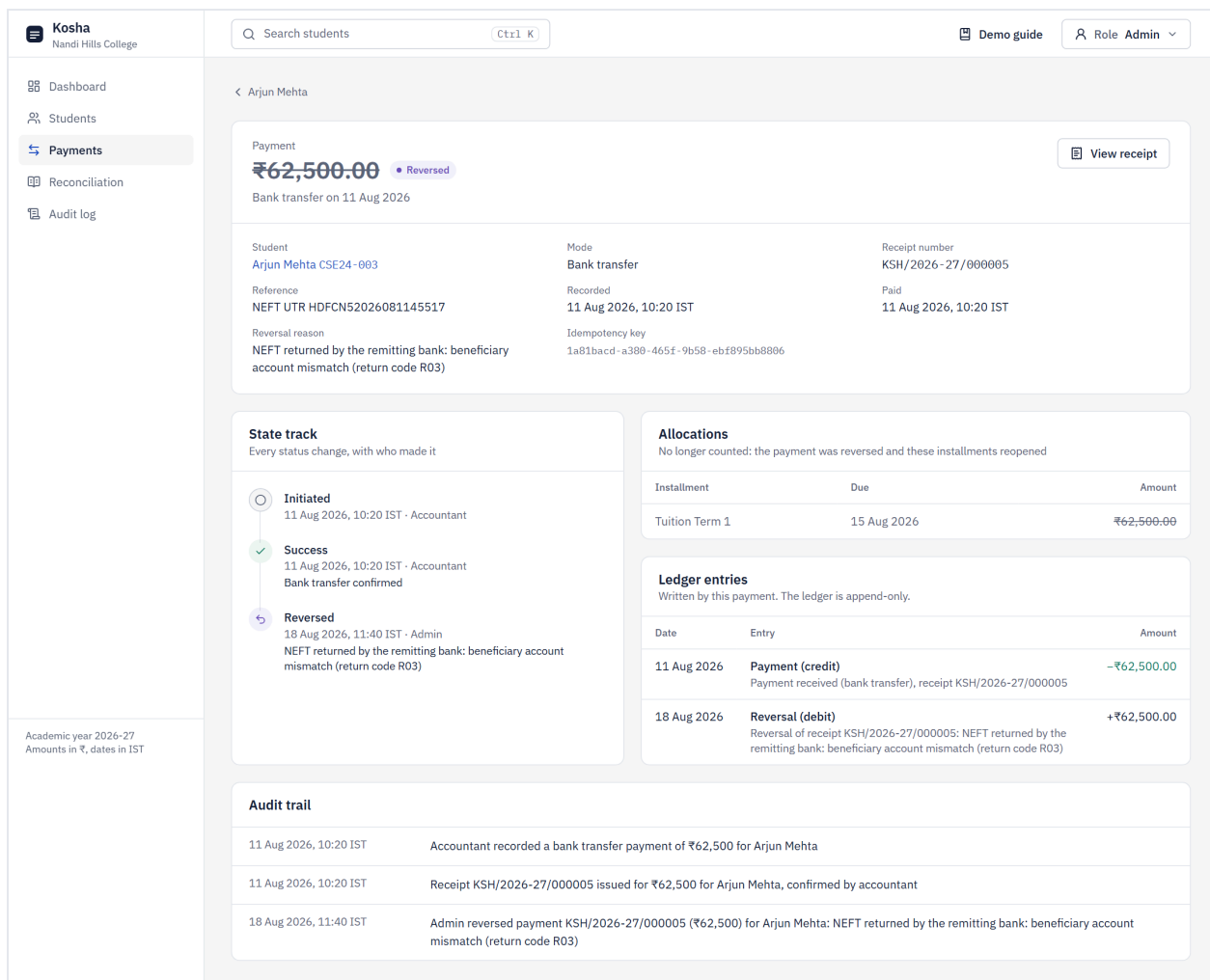


Figure 12. Payment detail: state track with actor and time, the installments it paid (struck through: it was reversed), the ledger entries it wrote, and the audit trail. Not shown: students list, payments list, concession and resolve drawers, the student-role view, the command palette and phone layouts.

12. AI features

Kosha has five AI features, built with OpenAI (`gpt-4.1-mini`) through LangChain and LangGraph. They are designed around one rule: **the AI never moves money**. It reads, investigates, explains and drafts; a person decides, and every decision goes through the same locked, audited database functions as the rest of the app.

Feature	What it does for the accounts office	Where
Reconciliation Copilot	Investigates each reconciliation exception (gateway record, payment history, duplicate payments, the file, other files) and proposes a resolution with evidence, a ready note and, when needed, a drafted message to the gateway.	Reconciliation run: <i>Investigate</i> , <i>Investigate all</i>
Ask Kosha	Answers questions about the fees in plain English ("Which BCA students are overdue by more than 30 days?"), with a linked table and follow-ups.	Top bar on every page, or Ctrl+J
Smart settlement import	Maps a settlement file in a gateway's own layout to Kosha's four columns, with a preview the accountant confirms.	Reconciliation upload
Messages to parents	Drafts reminders, overdue notices, balance explanations and payment confirmations in English, Kannada or Hindi, for WhatsApp or email.	Student page: <i>Message parent</i>
Daily finance brief	Opens the dashboard with the three things that most need attention today, each with the reason and a button to act.	Dashboard

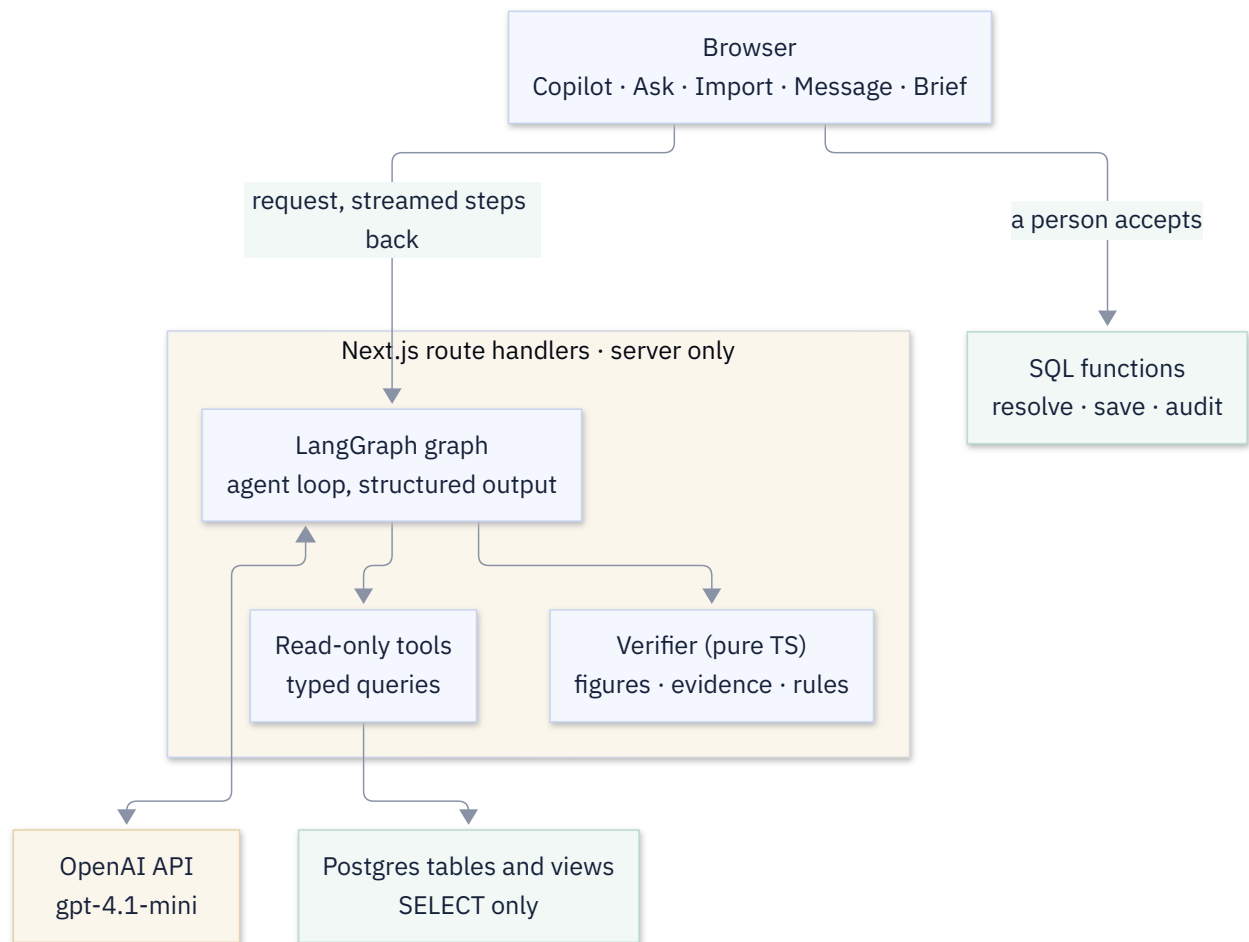


Figure 13. Where the AI sits. The OpenAI key is read only in server code. The model's tools can only read. The only writes are made by SQL functions after a person acts, or to store an AI result (a proposal, a brief, an audit line).

The pattern every feature follows

Language models are good at reading messy situations and writing clearly, and unreliable at arithmetic, at quoting figures exactly and at following rules every time. Each feature is therefore split so that **code owns the facts and the rules, and the model owns the judgement and the words**. Then code checks the model's work before anyone sees it.

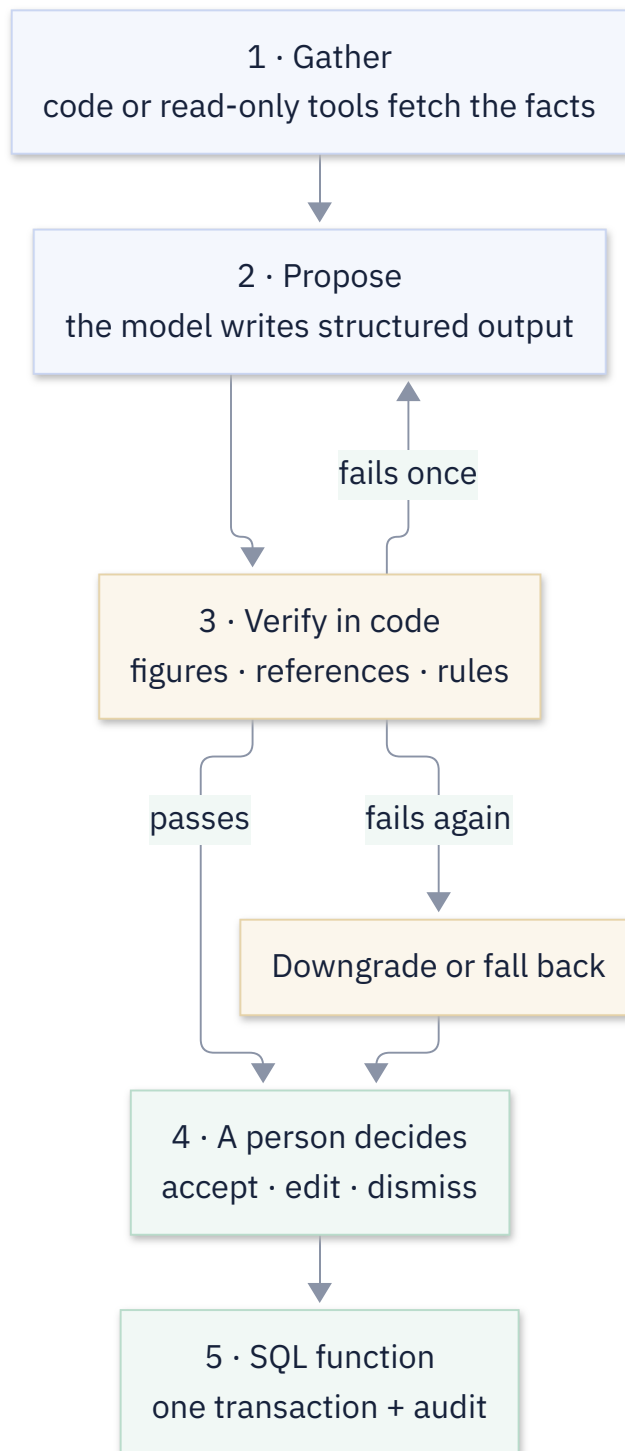


Figure 14. Gather, propose, verify, decide. Each feature is a LangGraph state machine with these nodes; the verify step is ordinary, unit-tested TypeScript.

Control	How it is enforced
No money moves without a person	Tools are read-only queries. Accepting a Copilot suggestion runs <code>decide_ai_investigation()</code> , which calls <code>resolve_recon_item()</code> in the same transaction; escalations cannot be accepted at all.
Every figure is real	A shared checker extracts every ₹ amount from the model's text and table cells; each must be one a tool returned (or the user typed). Totals, differences, percentages and date ranges are computed in code, not by the model.
Every claim has a source	Tools register evidence ids (<code>gateway:MGW...</code> , <code>student:CSE24-001</code> , <code>check:duplicates</code>); findings, table rows and sources must cite ids that exist, and the UI turns them into links.
Rules the model must follow	Allowed actions per bucket and payment status; a possible duplicate payment must be disclosed; actions in the brief must belong to their signal; messages must be in the chosen script, use ₹ digits and assume no gender.
Least data to OpenAI	Smart import sends only the header and at most 25 sample rows. Messages send the facts for one student. The key never reaches the browser bundle (checked after every build).
Auditable	Investigations, accept and dismiss decisions, drafted messages and briefs are stored or written to the audit log with who, when and which model.
Works without AI	With no <code>OPENAI_API_KEY</code> the AI buttons are hidden, and the daily brief is written by rules. Nothing else changes.

12.1 Reconciliation Copilot

Problem. A reconciliation run leaves exceptions: amounts that differ, money the gateway settled that is still pending here, payments recorded here but missing from the file. Each one takes an accountant several screens and a judgement call.

What it does. *Investigate* on an exception, or *Investigate all* for the run, streams the agent's steps live ("Asking the gateway about MGW7300000005", "Checking for a second payment of the same amount") and ends with a diagnosis: a headline, the root cause, a confidence, numbered findings linked to their records, the suggested action with an editable note, and for shortfalls a ready message to the gateway's support desk.

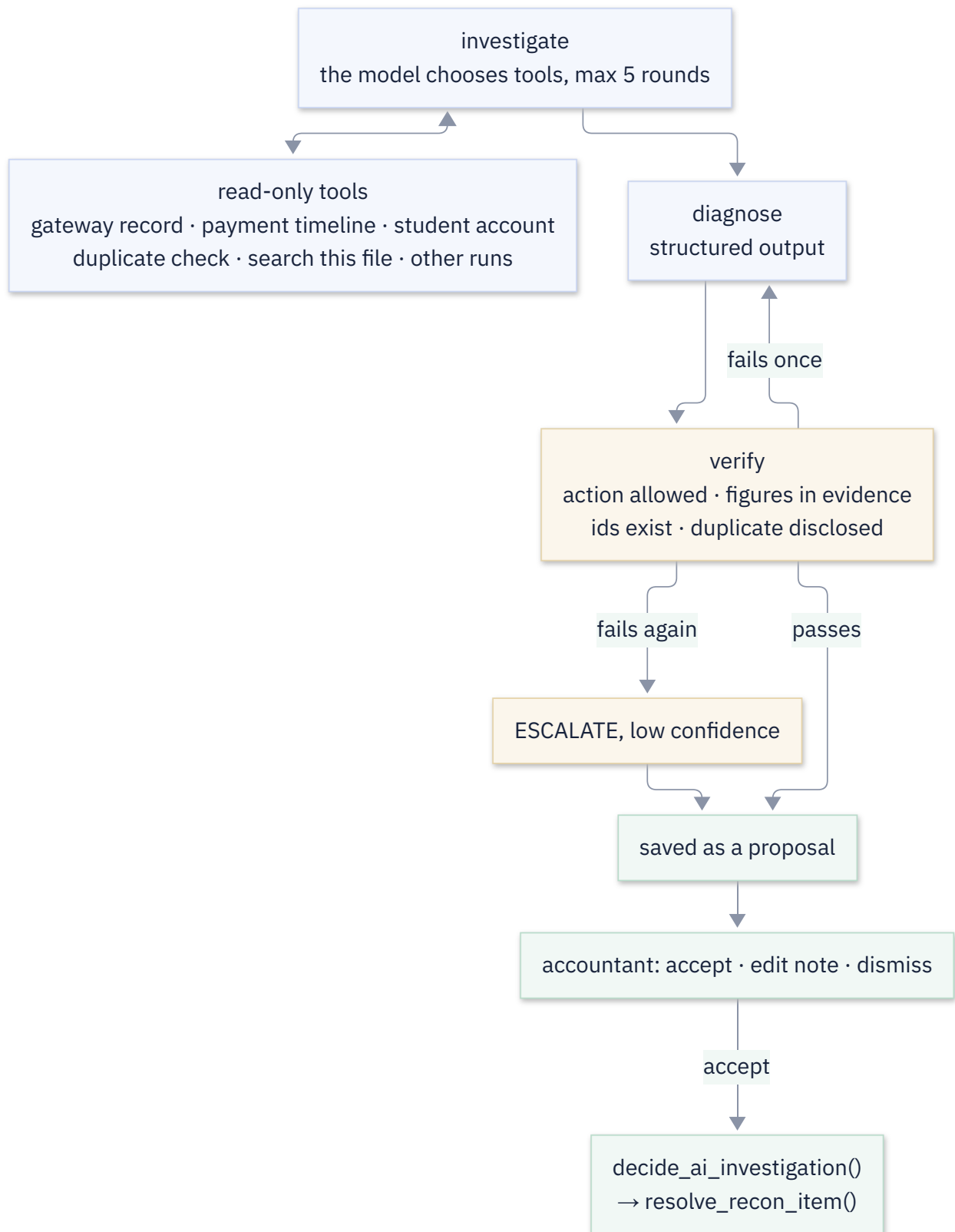


Figure 15. The Copilot's LangGraph. The agent decides which of six tools to call; code checks the diagnosis; a person applies it.

Case in the sample file	What the Copilot finds	Suggestion
Settled but pending here: Sneha Iyer, MGW7300000005, ₹86,500	The gateway's record says SUCCESS for ₹86,500; no second payment of the same amount. The UPI call timed out before the gateway confirmed.	Mark as paid (high)
Amount mismatch: MGW7300000023, file ₹62,000, recorded ₹62,500	The gateway captured ₹62,500 but settled ₹500 less. Drafts: "...the settlement file dated 13 Aug 2026 shows only ₹62,000 settled. Please clarify the reason for the ₹500 shortfall."	Escalate
Missing in settlement: MGW7300000039, ₹40,000	The gateway says SUCCESS; not in this file, not under a mistyped reference with the same amount, not in any other file.	Escalate
Staged in the eval: pending, and the parent then paid the same amount at the counter	Finds the second payment; marking the first as paid would count the money twice.	Escalate: possible duplicate

Reconciliation Copilot

MGW7300000005 · Sneha Iyer · Settled but pending here

In file

₹86,500

Recorded here

₹86,500

Payment now

pending

WHAT THE COPILOT CHECKED

✓ Asking the gateway about MGW7300000005

Gateway says success for ₹86,500

✓ Checking for a second payment of the same amount

No second payment of the same amount

🔍 Diagnosis

Gateway confirmed after we timed out

high confidence

Payment MGW7300000005 was successful but still marked pending here without duplicates.

1. Gateway says success for ₹86,500 for MGW7300000005.

Gateway record MGW7300000005

2. No second payment of the same amount found for Sneha Iyer (roll no BCA26-002).

Duplicate payment check

✓ Checked against the database

1 rupee figure and every cited record were found in what the tools returned. The suggested action is allowed for this item.

SUGGESTED ACTION

Mark as paid

The gateway confirms success for the full amount and no duplicate payment exists, so mark this payment as paid.

Note saved on the item (you can edit it)

Checked gateway record and duplicates; payment confirmed successful.

Investigated 29 Sep 2026, 19:38 IST · 2 lookups · 5.7s · 5,255 tokens · gpt-4.1-mini

🔄 Investigate again

Dismiss

Accept: mark as paid

Figure 16. A Copilot suggestion: what it checked, the diagnosis with linked evidence, the database check, and *Accept: mark as paid*.

Reconciliation Copilot

MGW7300000023 · Lakshmi Narayanan · Amount mismatch

In file

₹62,000

Recorded here

₹62,500

Difference

-₹500

WHAT THE COPILOT CHECKED

✓ Asking the gateway about MGW7300000023

Gateway says success for ₹62,500

✓ Checking for a second payment of the same amount

No second payment of the same amount

🔍 Diagnosis

Gateway settled less than was paid

high confidence

Gateway recorded ₹62,500 but settlement file shows ₹62,000, a ₹500 shortfall at settlement for Lakshmi Narayanan CSE26-009.

1. Gateway says success for ₹62,500 for MGW7300000023 on 12 Aug 2026, matching our recorded amount.

Gateway record MGW7300000023

2. Settlement file shows ₹62,000 settled on 13 Aug 2026, ₹500 less than gateway captured.

Amount mismatch · MGW7300000023

3. No duplicate payments of ₹62,500 found for this student to explain difference.

Duplicate payment check

✓ Checked against the database

13 rupee figures and every cited record were found in what the tools returned. The suggested action is allowed for this item.

SUGGESTED ACTION

Escalate to a person

The gateway captured ₹62,500 but settled ₹62,000, indicating a shortfall that must be clarified with the gateway.

DRAFT MESSAGE TO THE GATEWAY

Copy

Dear Support, we have a payment with reference MGW7300000023 for student Lakshmi Narayanan (roll no CSE26-009). The gateway captured ₹62,500 on 12 Aug 2026, but the settlement file dated 13 Aug 2026 shows only ₹62,000 settled. Please clarify the reason for the ₹500 shortfall and advise on resolution. Thank you. Accounts Office.

COULD BE WRONG IF

Gateway may have adjusted fees or charged a fee deducted from settlement.

Possible data entry or timing error in settlement file.

Gateway record might be updated later with corrected amount.

Investigated 29 Sep 2026, 19:37 IST · 2 lookups · 9.1s · 5,723 tokens · gpt-4.1-mini

🔄 Investigate again

Dismiss

Resolve by hand

Figure 17. For the ₹500 shortfall it escalates and drafts the message to the gateway. An escalation can only be resolved by hand.

Reconciliation

settlement_sample.csv

Run by Admin on 29 Sep 2026, 19:37 IST · covers payments made 9 Aug 2026 to 21 Sep 2026

Rows in file	Matched	Amount mismatch	Settled but pending here	Recorded here, missing in settlement
36	33 ₹20,35,375	1 ₹62,000	2 ₹1,62,500	1 ₹40,000

🔍 Reconciliation Copilot

4 of 4 open exceptions have a suggestion ready. It checks the gateway, the student's account and the file, then suggests a resolution. You decide.

Matched 33

Amount mismatch 1

Settled but pending here 2

Recorded here, missing in settlement 1

The gateway settled these, but they are still pending here. Mark as paid to confirm the payment, issue the receipt and allocate it.

File ₹1,62,500

Gateway ref	Student	Settled	In file	Recorded here	Payment now	Resolution
MGW7300000005	Sneha Iyer	22 Sep 2026	₹86,500.00	₹86,500.00	Pending	🔍 Mark as paid high Mark as paid Mark reviewed
MGW7300000008	Aishwarya Gowda	20 Sep 2026	₹76,000.00	₹76,000.00	Pending	🔍 Mark as paid high Mark as paid Mark reviewed

Figure 18. After *Investigate all*: every open exception carries a suggestion chip with its confidence, next to the manual actions.

```
Code: lib/ai/recon-copilot/ (graph, tools, prompts, schema, verify),
supabase/migrations/004_ai.sql (ai_investigations, save_ai_investigation,
decide_ai_investigation), streaming route app/api/ai/recon-items/[id]/investigate,
components/ai/copilot-drawer.tsx.
```

12.2 Ask Kosha

Problem. Questions like "who in second-year B.Com owes more than ₹50,000?" or "how did UPI collections compare with last week?" today need filters, exports and a spreadsheet.

What it does. A chat panel on every page. The answer leads with the number, shows a table whose rows open the student or payment, lists its sources, suggests follow-ups, and says how many figures were checked. The conversation carries over, so "only the BCA ones" or "how much do they owe in total?" work.

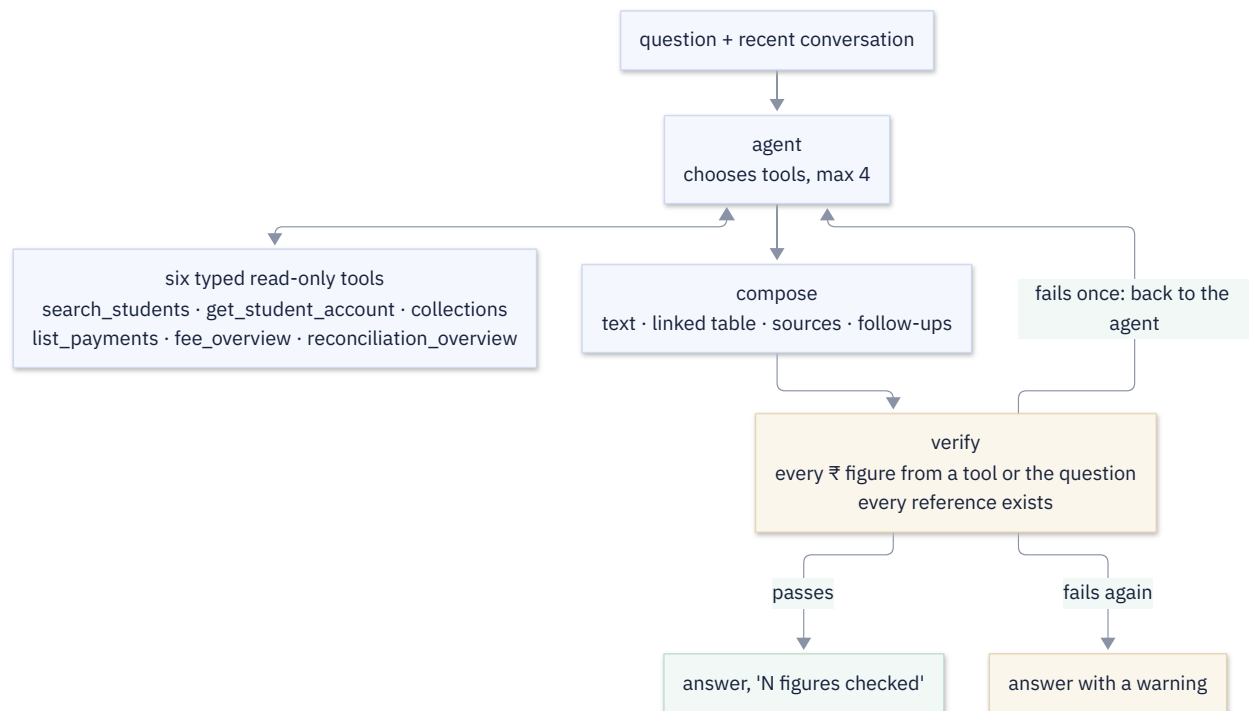


Figure 19. Ask Kosha's graph. A failed check sends the model back to the tools, not just back to rewording, so a total it tried to add up itself gets looked up instead.

Design decisions

- **Typed tools, not text-to-SQL.** The model fills validated parameters (course, status, minimum amount, date range, grouping); it can only read what these six functions return and can never write.
- **Arithmetic and dates in code.** Tools return totals for all matches, per-group comparisons and percentage changes. "Last week" is resolved in code (Monday to Sunday) and the exact ranges are given to the model, after the same question once gave two different weeks.
- **Ties are never cut.** "Who owes the most?" lists all three students on ₹1,73,000.
- **Stateless follow-ups.** The browser sends the recent turns (including the previous table) with each question, so it works on serverless hosting without a session store.
- **Read-only and on topic.** "Record a cash payment..." is refused with a pointer to the right screen; questions unrelated to the fees are declined.

Ask Kosha

Questions about fees, answered from the ledger. Read-only.

Which students owe more than ₹1,00,000 and are overdue?

There are 12 students who owe more than ₹1,00,000 and are overdue. Their outstanding amounts range from ₹1,07,000 to ₹1,73,000, with overdue amounts between ₹26,500 and ₹84,000. They are from courses BCA, CSE, and BCOM, mostly in years 1 to 4, and have overdue durations of 45 days.

Name	Roll No	Course	Year	Outstanding	Overdue
Sneha Iyer	BCA26-002	BCA	1	₹1,73,000	₹84,000
Meera Pillai	BCA25-003	BCA	2	₹1,73,000	₹84,000
Divya Hegde	BCA24-005	BCA	3	₹1,73,000	₹84,000
Suresh Babu	CSE24-011	CSE	3	₹1,58,000	₹42,000
Aishwarya Gowda	BCOM25-004	BCOM	2	₹1,52,000	₹74,000
Arvind Swamy	BCOM26-014	BCOM	1	₹1,52,000	₹74,000
Rohan Kulkarni	CSE24-001	CSE	3	₹1,42,500	₹26,500
Swati Agarwal	CSE23-012	CSE	4	₹1,35,000	₹64,000
Rajat Bansal	CSE24-015	CSE	3	₹1,35,000	₹64,000
Arjun Mehta	CSE24-003	CSE	3	₹1,30,000	₹62,500
Gayatri Iyengar	BCA25-018	BCA	2	₹1,22,000	₹33,000
Manish Tiwari	BCOM26-008	BCOM	1	₹1,07,000	₹29,000

30 figures checked against the ledger · 1 lookup · 17.2s

Show only BCA students who owe more than ₹1,00,000 and are overdue

How much total overdue amount do these students owe?

List students who owe more than ₹1,00,000 but are not overdue

only the BCA ones

There are 4 BCA students who owe more than ₹1,00,000 and are overdue. Their outstanding amounts range from ₹1,22,000 to ₹1,73,000, with overdue amounts between ₹33,000 and ₹84,000. They have been overdue for 45 days and some have pending payments.

Name	Roll No	Year	Outstanding	Overdue	Days Overdue
Sneha Iyer	BCA26-002	1	₹1,73,000	₹84,000	45
Meera Pillai	BCA25-003	2	₹1,73,000	₹84,000	45
Divya Hegde	BCA24-005	3	₹1,73,000	₹84,000	45
Gayatri Iyengar	BCA25-018	2	₹1,22,000	₹33,000	45

14 figures checked against the ledger · 1 lookup · 7.3s

Which BCA students have pending payments?

How much do these BCA students owe in total?

Show overdue students in CSE owing more than ₹1,00,000

e.g. Which BCA students are overdue by more than 30 days?

Enter to ask · Shift+Enter for a new line · Read-only: it cannot change anything

New chat

Figure 20. Ask Kosha: a question with a threshold, then the follow-up "only the BCA ones". Names link to the statements; each answer states how many figures were checked against the ledger.

Code: `lib/ai/ask/` (tools, graph, periods, schema, verify, prompts), route `app/api/ai/ask`, `components/ai/ask-kosha.tsx`.

12.3 Smart settlement import

Problem. Real settlement reports never use Kosha's four column names. They have "Txn Reference", gross, MDR, GST and net amounts, a payout UTR, a transaction date next to the settlement date, and DD/MM/YYYY 12-hour times. Retyping headers is slow and error-prone, and choosing the net amount would turn every card payment into a mismatch.

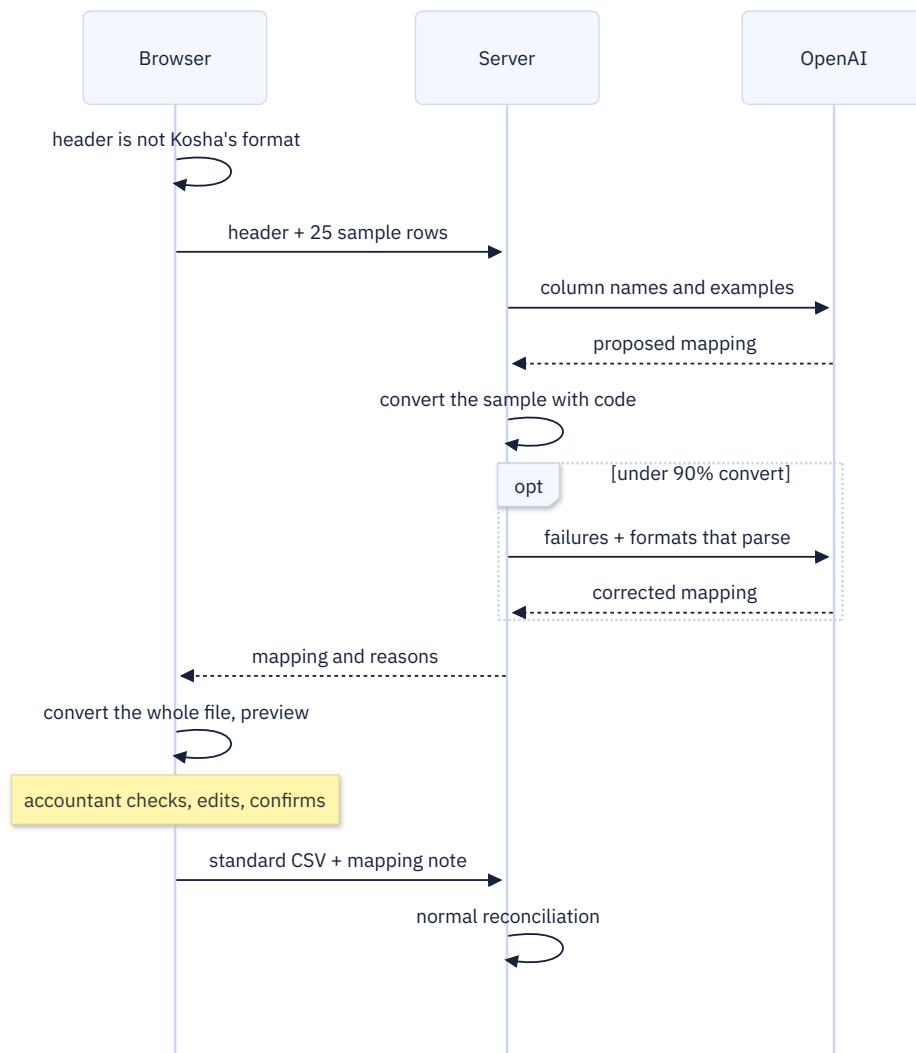


Figure 21. Smart import. The AI only chooses which column is which; deterministic code does every conversion, so a wrong choice shows up as rows that fail to convert, not as silently wrong numbers.

- **Traps it has to avoid in the sample:** Net vs Gross amount, Txn Date vs Settlement Date, Settlement UTR vs Txn Reference. The eval requires all three right.
- **Every status value is seen.** Sampling includes at least one row per distinct value of each category-like column, and the preview lists every status in the whole file; reconciling is blocked until each one is mapped.
- **Day/month ambiguity.** If no date has a day above 12, DD/MM and MM/DD both convert every row with different dates, so the preview requires the accountant to confirm the format.
- **Remembered layouts.** A confirmed mapping can be saved for files with the same columns, and the run page records which mapping was used and who confirmed it.


gateway_report_sep_2026.csv uses a different layout
 Kosha needs a gateway reference, amount, status and settlement date for each row.
 The import assistant matched them to this file's columns. Check each one before reconciling.

Kosha needs	From this column	Why
Gateway reference The transaction reference Kosha stored for the payment.	Txn Reference e.g. MGW7300000006, MGW7300000012	This column contains the gateway's transaction reference in the required format (e.g., MGW7300000012).
Amount Gross amount the parent paid, before gateway fees.	Gross Amount (Rs) in rupees e.g. 1,12,500.00, 1,35,000.00	This column shows the gross amount paid by the payer, matching the requirement for amount_inr.
Status Whether the transaction settled.	Txn Status Captured → Settled Failed → Failed e.g. Captured, Captured	Mapped 'Captured' to SUCCESS and 'Failed' to FAILED as per instructions.
Settled on When the gateway paid the money to the college.	Settlement Date DD/MM/YYYY e.g. 10/08/2026 11:00 AM, 11/08/2026 11:00 AM	Settlement Date shows when the gateway settled the money; time is ignored, date format is DD/MM/YYYY as per Indian default.

- 'Settlement UTR', 'Txn Date', 'Customer Name', 'Payment Method', 'MDR', 'GST on MDR', and 'Net Settled (Rs)' columns are ignored as they are not required for Kosha mapping.
- Status values are limited to 'Captured' and 'Failed' in this data, so no other mappings needed.
- Amounts are in rupees with commas and decimals, no conversion needed.


37 of 37 rows convert cleanly

What Kosha will reconcile (first 4 rows)

gateway_ref	amount_inr	status	settled_at
MGW7300000006	112500.00	SUCCESS	2026-08-10T11:00:00+05:30
MGW7300000012	135000.00	SUCCESS	2026-08-11T11:00:00+05:30
MGW7300000023	62000.00	SUCCESS	2026-08-13T11:00:00+05:30
MGW7300000003	51875.00	SUCCESS	2026-08-14T11:00:00+05:30

☒ Remember this mapping for files with the same columns


high confidence
Sent to OpenAI: the column names and 25 sample rows, not the whole file · 2.8s · gpt-4.1-mini

Figure 22. The import preview for the gateway-layout sample: each choice with the assistant's reason in an editable dropdown, status meanings, notes on the ignored columns (including Net Settled), the result on all 37 rows and what will be reconciled.

Code: `lib/domain/settlement-mapping.ts` (converter, pure and unit tested), `lib/ai/settlement-mapper.ts` (graph), `route app/api/ai/settlement-mapping`, `components/reconciliation/smart-import.tsx`. Sample: `public/samples/gateway_report_sep_2026.csv`.

12.4 Messages to parents

Problem. Reminders and overdue notices are typed by hand, often in the parent's language, and a wrong amount in a fee message damages trust.

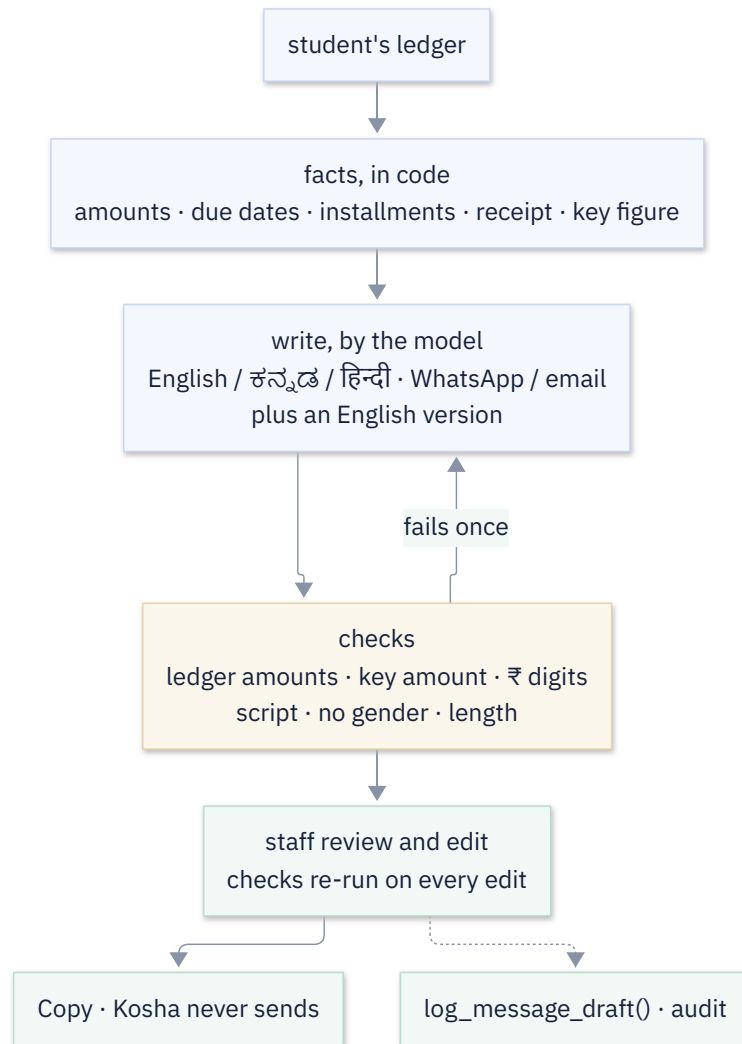


Figure 23. Drafting a message. The checks that run on the model's draft also run in the browser while staff edit, so a wrong amount typed by a person is flagged too.

- **Four purposes**, available only when they apply: upcoming due, overdue notice (no penalties or threats; the college charges no late fees), balance explained, payment received (with the receipt number and what it paid for).
- **Kannada and Hindi** in natural, everyday language, always with an English version so staff who do not read the script can check what it says.
- **No assumed gender.** The first drafts wrote "your daughter" and "आपके पुत्र" from the names alone; Kosha does not record gender, so the prompt forbids it and a check fails any draft with gendered words in any of the three languages.
- **Amounts must be checkable:** written with ₹ and 0-9 digits, never in words or native digits.

Message to parent

Sneha Iyer · BCA26-002

Message

Upcoming due

Overdue notice

Balance explained

Payment received

Language

English

ಕನ್ನಡ

ಇನ್ನಿ

For

WhatsApp

Email

Message

ಪೂಜ್ಯಪರ,

ನಂದಿ ಹಿಲ್ಸ್ ಕಾಲೇಜಿನ ಖಾತೆಗಳ ಕಚೇರಿಯಿಂದ ತಿಳಿಸುವುದು: ನಿಮ್ಮ ಮಗು Sneha Iyer (ರೋಲ್ ನಂ: BCA26-002) ಅವರ 1ನೇ ವರ್ಷದ Bachelor of Computer Applications ಕೋರ್ಸ್ ಶುಲ್ಕಗಳಲ್ಲಿ ₹84,000 ಮೊತ್ತದ ಬಾಕಿ ಶುಲ್ಕಗಳು 15 Aug 2026 ರಿಂದ ಬಾಕಿ ಇವೆ.

ಈ ಮೊತ್ತದಲ್ಲಿ, Tuition Term 1 ₹38,000, Hostel Term 1 ₹45,000 ಮತ್ತು Library Term 1 ₹1,000 ಸೇರಿವೆ.

ದಯವಿಟ್ಟು ಶೀಘ್ರದಲ್ಲೇ ಈ ಬಾಕಿ ಶುಲ್ಕಗಳನ್ನು ಪಾವತಿಸಿ ಅಥವಾ ಯಾವುದೇ ಪ್ರಶ್ನೆಗಳಿದ್ದರೆ ಖಾತೆಗಳ ಕಚೇರಿಯನ್ನು ಸಂಪರ್ಕಿಸಿ.

Checked against the ledger

✓ All 8 amounts are from the ledger.

✓ Every amount is written with ₹.

✓ Refers to the student by name or as your ward.

✓ ₹84,000 is in the message.

✓ Written in Kannada script.

✓ An English version is included for checking.

✓ 615 characters.

▼ What this says, in English

Dear Parent,

From the Accounts Office of Nandi Hills College: Your ward Sneha Iyer (Roll No: BCA26-002) has overdue fees of ₹84,000 since 15 Aug 2026 for the 1st year Bachelor of Computer Applications course.

This amount includes Tuition Term 1 ₹38,000, Hostel Term 1 ₹45,000, and Library Term 1 ₹1,000.

Please pay the outstanding fees soon or contact the Accounts Office if you have any questions.

Payments can be made at the college accounts counter (cash, card, or UPI) or online via UPI or card through the student portal. Please quote the roll number.

Thank you,
Accounts Office, Nandi Hills College

Drafted in 5.9s · 1,343 tokens · gpt-4.1-mini · recorded in the audit log

Kosha does not send messages. Copy it into WhatsApp or email.

↻ Draft again

📄 Copy message

Figure 24. An overdue notice in Kannada for WhatsApp: every check passed, with the English version below. The drafting is recorded in the audit log; Kosha does not send it.

Code: `lib/ai/parent-message/` (facts, draft graph, verify), route `app/api/ai/parent-message`, `supabase/migrations/005_ai_messages.sql`, `components/ai/parent-message-drawer.tsx`.

12.5 Daily finance brief

Problem. The dashboard shows everything; an accountant arriving in the morning needs to know what to do first.

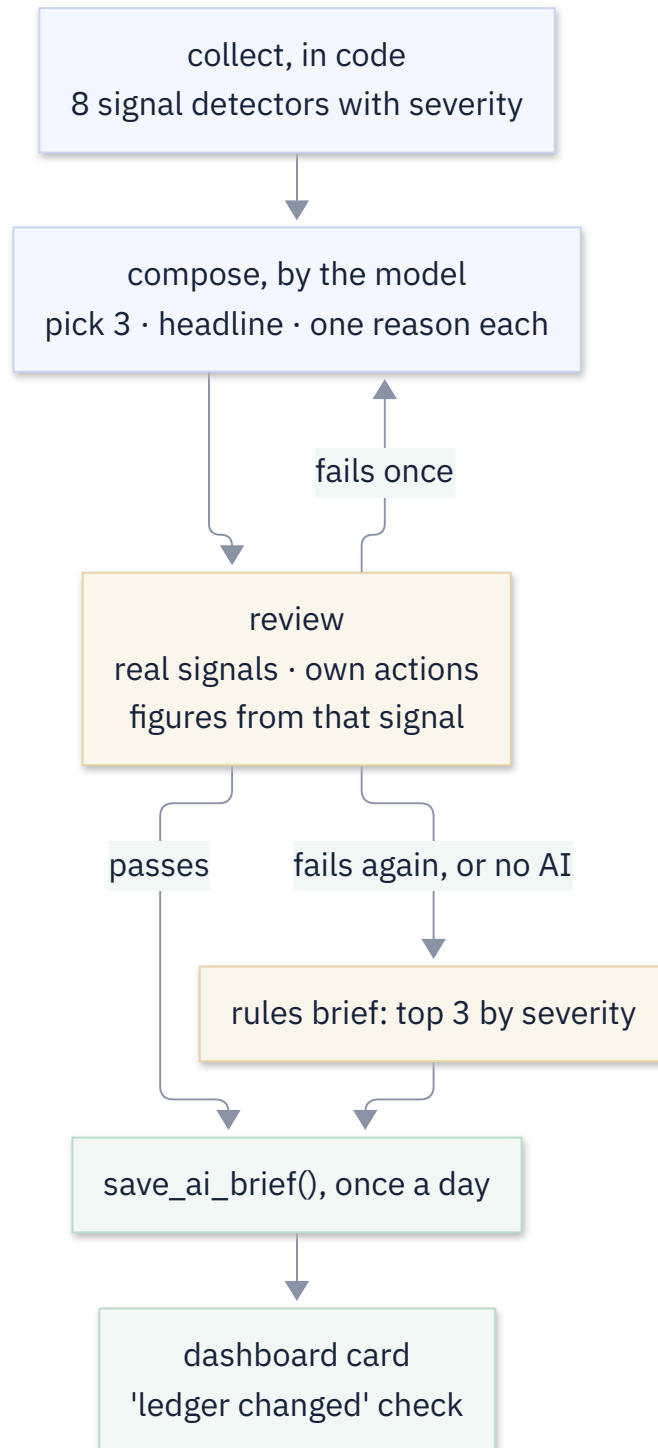


Figure 25. The brief. Titles, figures, examples and links are the code's; the model's words are only the headline and the reasons. With no AI, a rules brief takes its place.

- **Signals are counted once.** The same exception in two runs of the same file counts once; the link goes to the newest run.

- **It knows when it is out of date.** A fingerprint of the signals is compared on every dashboard load; after a payment the card says "The ledger has changed since this was written" and highlights Refresh.
- **Calm tone.** Early briefs said "to secure ₹8,81,000" and "significant revenue loss"; the prompt now asks for plain, calm wording, and the eval's quiet day checks that no urgency is invented.

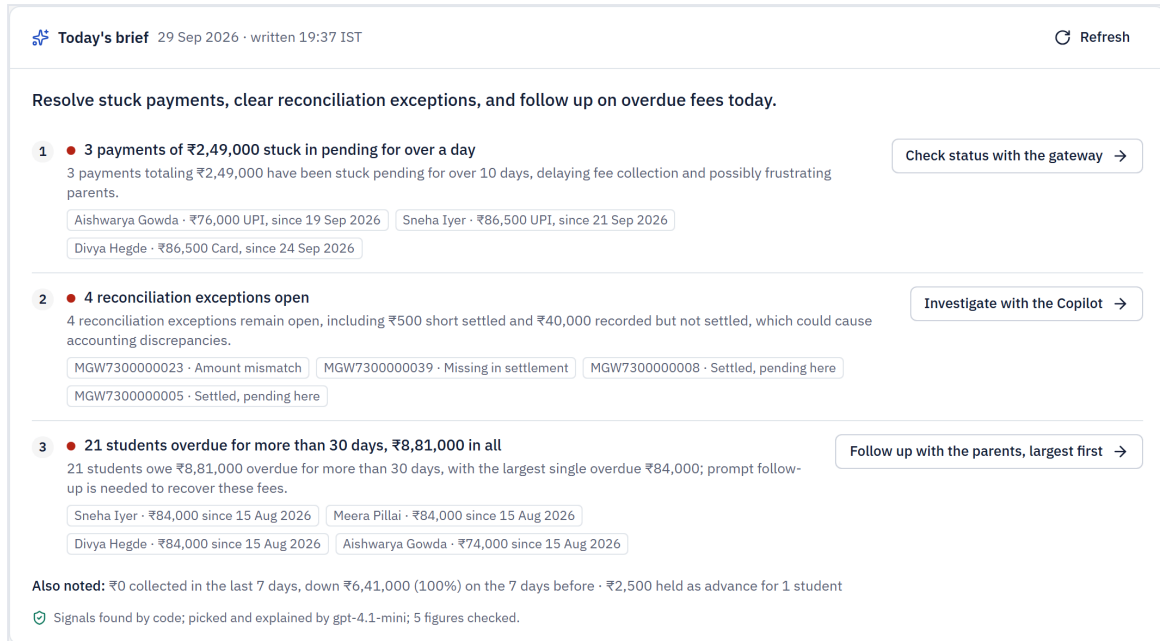


Figure 26. Today's brief on the demo ledger: stuck payments first, then open exceptions, then long-overdue students, each with example records and an action; the rest under "Also noted".

Code: `lib/ai/brief/` (signals, brief graph), `lib/data/brief.ts`, route `app/api/ai/brief`, `supabase/migrations/006_ai_brief.sql`, `components/ai/daily-brief.tsx`.

12.6 How the AI features are tested

Two layers. **Unit tests** cover every deterministic part (verifiers, date periods, the settlement converter, the signal detectors), including the failure cases the verifiers exist to catch: a made-up amount, an unknown evidence id, a forbidden action, an undisclosed duplicate, an amount written in words or Kannada digits, a gendered word in Hindi. **Evaluations** (`npm run eval:ai`) run the real model on cases whose right answer is known, with the answer computed independently from the database, not by the feature's own tools.

Evaluation	Cases	Result
Reconciliation Copilot	Settled but pending → mark as paid; ₹500 shortfall → never mark as paid, cause "settlement short", gateway message mentions ₹500; missing → never mark as paid, file searched; staged double payment → possible duplicate, not marked paid.	4/4
Ask Kosha	Number of overdue students; college-wide overdue total; the top debtor (with ties); last week's collections; everyone with a pending payment; a two-turn follow-up; refusing to delete a payment.	8/8
Smart import and messages	The gateway layout (gross, settlement date, reference, DD/MM/YYYY); a layout in paise with ISO dates and no status column; four messages in Kannada, Hindi and English for WhatsApp and email.	6/6
Daily brief	The demo ledger; a busy day where stuck payments and a shortfall must lead, not the advance; a quiet day with no invented urgency.	3/3

The evaluations found real problems: a duplicate check blind to a payment made 8 days later, "last week" meaning two different weeks, a tie cut to one student, YYYY/MM/DD chosen for dashed dates, and "who owes the most?" answered from only the DUE students, with correct figures that the verifier passed. **The verifier catches invented numbers; the evaluations catch wrong interpretations.** All are fixed and logged in `AI_LOG.md`.

12.7 Cost and speed

All features use `gpt-4.1-mini` at temperature 0, with a 45-second timeout and two retries (set by `OPENAI_MODEL`). Each use records its token count.

Feature	Typical time	Tokens per use	Notes
Copilot, one exception	6 to 8 s	5,000 to 6,000	2 to 3 tool calls, streamed live.
Ask Kosha, one question	4 to 8 s	5,000 to 8,000	Up to 17 s for long tables.
Smart import	about 3 s	about 1,600	Once per new layout.
Message to a parent	2 to 6 s	1,300 to 2,000	Includes the English version.
Daily brief	about 4 s	about 1,200	Once a day.

13. Limitations and future work

Out of scope by design

- **A real payment gateway** (webhooks with signature checks, status polling, settlement file download).
- **Real authentication and per-user row-level security**, replacing the role cookie; per-student access for parents.
- **Late fees** and interest on overdue installments.
- **A fee-structure editor**: fee heads, amounts and due dates per course and term.
- **Multiple institutions** (tenancy) and multiple academic years side by side.
- **Email and SMS**: receipts, reminders before due dates, notices of reversals.
- **Partial refunds**; today a payment can only be reversed in full.

Known limitations

- Anyone with the demo link can switch to Admin and reset the data. Acceptable for a demo; the first thing real auth removes.
- Reconciliation loads all gateway payments into memory for matching, fine for thousands of rows; at scale the window filter should move into SQL.
- Settlement fees (MDR) are not modelled, so a real gateway's net settlements would all show as mismatches until fees are added as a column.
- A settled-but-pending item that is later confirmed through check status stays open until someone resolves it (resolving it then records "already confirmed").
- Receipt numbers come from a sequence and can have gaps after a rolled-back transaction.
- A receipt lists the installments its payment pays today; if an advance is applied later, an old receipt gains a line. A receipt snapshot table would freeze it.
- The dashboard is computed per request; a materialised daily summary would be needed for many thousands of students.

AI limitations

- The verifiers catch invented figures and references, not a wrong interpretation with real figures (a wrong filter, a misread question). The evaluations cover the known cases; a larger evaluation set, run on every change to a prompt or model, is the next step.
- The model's answers can vary slightly between runs even at temperature 0; decisions that matter always pass through a person.
- Features depend on OpenAI being reachable. Without it, the buttons are hidden and the daily brief falls back to rules.
- Sample rows sent for smart import can include payer names. A deployment would need a data-processing agreement with the provider, or an option to mask names before sending.
- Kannada and Hindi drafts are checked for script, figures and gender, not fluency, and the one-investigation-per-item guard is in server memory (several servers would need it in the database).

14. AI usage report

Drafted only from the entries in `AI_LOG.md` (30 entries, written as the work happened; entries 15 to 30 cover the AI features). Highlighted text is for the candidate to complete in their own words.

AI tool used

Claude Code (Anthropic's coding agent, model Claude Opus 5.5), running in the Claude desktop app with access to the project folder, a shell, a browser pane and Playwright. *[Darshan: confirm, and add any other tools you used]*

What I asked AI to do

I wrote a detailed project brief (stack, domain rules, design system, phases with a definition of done for each) and asked the assistant to build it phase by phase: database schema, money functions and seed; domain layer, API and tests; design system and screens; payments, receipts and reconciliation; end-to-end tests, README and this document. After each phase I reviewed the result and approved the next. *[Darshan: add your own words on how you directed and reviewed the work]*

Most useful prompt

[Darshan: add the prompt you found most useful, for example the brief's rule "the database is the source of truth for all money rules; every money-changing operation is a single PostgreSQL function", or the instruction to log every mistake in AI_LOG.md]

Code generated by AI

Most of the code was generated by the assistant from the brief: the SQL migrations and functions, the seed, the domain functions and their tests, the API routes, the components and pages, the Playwright tests and the documentation build. *[Darshan: state the proportion in your own words]*

Code I modified

[Darshan: list what you changed or decided yourself, e.g. choices you made at the phase reviews]

AI output that was wrong, how I identified it, how it was fixed

What was wrong	How it was identified	How it was fixed
Seed payment dates only took 4 distinct values (<code>(i*11) % 44</code>).	Grouping payments by day in a dry run on an in-memory Postgres.	Coprime step <code>(i*17) % 45</code> ; re-verified.
Row types clashed with supabase-js inference (embedded relations typed as arrays).	<code>npm run typecheck</code> errors.	Query helper returns the row type declared at the call site.

What was wrong	How it was identified	How it was fixed
Payment search ORed across an embedded table, which PostgREST cannot do.	Re-reading the query before running it.	Match students first, then OR on ids.
Cached demo-student id would go stale after a reset.	Self-review of the reset route.	Cache removed.
Awkward audit sentences and "already success" error text.	Reading API smoke-test output.	Rewrote the copy; "₹" instead of "Rs" in SQL messages.
Tailwind opacity modifiers on CSS-variable colours generated nothing.	Self-review before the first screenshot.	RGB channel variables with <code><alpha-value></code> .
Dates rendered "24 Sept 2026", then "05 Jan 2027".	Dashboard screenshot; then a new unit test.	Fixed month names, unpadded day.
Layout issues: misaligned fee-head bars, approver in the monospaced column, phone overflow, a sentence entirely red, a redundant advance sentence, a palette loading flash.	Reading 1440 px and 390 px screenshots.	Fixed column, secondary line, phone layouts, split sentence, prefetch.
Invisible combining characters in a regex after a scripted edit.	An odd diff line, confirmed with <code>od -c</code> .	<code>\p{Diacritic}</code> .
Select chevron written as a Tailwind class containing spaces, so no dropdown showed its arrow.	A Playwright locator error printed the split classes.	Moved to a CSS class.
Toasts covered the drawer's submit; weak selected state; truncated "Bank transfer"; receipt spilled onto a second page; an ALL-CAPS title against the brief.	Screenshots, a PDF render of the receipt, re-reading the brief.	Toasts bottom-left, ink fill for selection, 2×2 on phones, tighter print spacing, sentence case.
Test scripts waited on the wrong signal or read the wrong element, reporting failures in working features.	Results contradicted the screen; checked through the API.	Wait for the specific toast or dialog; filter alerts by text.
Dashboard: duplicated open items after a repeated upload, overflowing badge, mismatched counts, a smoothed trend line, a blank chart in one capture.	Reading dashboard screenshots.	De-duplication, short labels, clear counts, straight segments, initial chart size.
Copilot findings cited the exception itself; a search tool called 36 rows "similar" because references are sequential.	Reading the first live investigations.	Evidence ids for every check; near-identical reference counts only with the same amount.

What was wrong	How it was identified	How it was fixed
The duplicate-risk check accepted "no duplicate"; the duplicate search looked only 7 days back, and the eval passed anyway.	A unit test; the eval's summary table said pass: false.	Risk must be in risks or next steps; any later same-amount payment counts; stricter eval judge.
A LangGraph node named like a state field (twice, in AI-2 and AI-3).	The graph refused to compile on the first call.	Renamed; every graph checked.
"Last week" meant two different weeks on the same day; a follow-up total added up by the model; a tie cut to one student; placeholder cells.	UI screenshots and the Ask Kosha eval.	Date ranges resolved in code; failed checks go back to the tools; ties kept; per-group comparisons.
"Who owes the most?" answered from the DUE students only, with correct figures.	The full eval run (7/8), then asking three times.	Filter description rewritten; 8/8 twice.
The import sample missed the only "Failed" row; a wrong day/month choice converted without errors; YYYY/MM/DD chosen for dashed dates twice.	Reading the proposal, a unit test, the AI-3 eval.	Sampling covers every status value; ambiguity confirmation; retry told which formats parse.
Drafted messages guessed gender ("your daughter", "आपके पुत्र"); early briefs used alarmist wording.	Reading the drafts and the first brief.	Gender rule plus a check in three languages; calmer prompt and a quiet-day eval.
<i>[Darshan: add anything you caught yourself during review that is not in the log]</i>		